



LakeDB

One workspace for SQL, data and database operations.

MySQL · MariaDB · SQLite

ENGINEERING OVERVIEW

A practical walkthrough of connection safety, SQL workflows, local tooling and review-first AI.

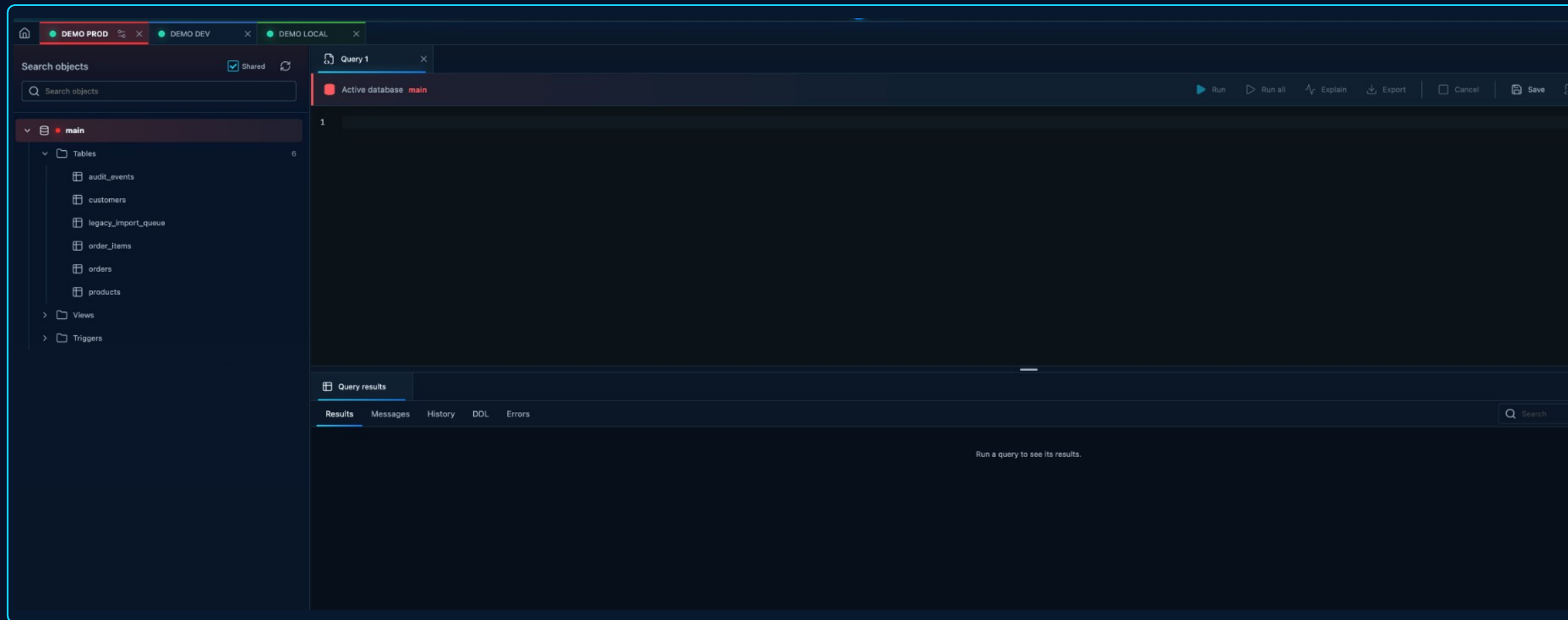
The screenshot displays the LakeDB interface with a SQL query editor and a results table. The query is a SELECT statement joining customers and orders, calculating order counts and lifetime values. The results table shows customer data with columns for customer_ref, full_name, segment, country, order_count, lifetime_value, and last_order_at.

```
1 SELECT
2   c.customer_ref,
3   c.full_name,
4   c.segment,
5   c.country,
6   COUNT(o.id) AS order_count,
7   ROUND(COALESCE(SUM(o.total_amount), 0), 2) AS lifetime_value,
8   MAX(o.ordered_at) AS last_order_at
9 FROM customers c
10 LEFT JOIN orders o ON o.customer_id = c.id
11 GROUP BY
12   c.id,
13   c.customer_ref,
14   c.full_name,
15   c.segment,
16   c.country
17 ORDER BY
18   lifetime_value DESC,
19   c.full_name
20 LIMIT 8;
```

customer_ref	full_name	segment	country	order_count	lifetime_value	last_order_at
CUS-1001	Maya Chen	Enterprise	US	2	9820	2026-02-14 14:15:00
CUS-1003	Elena Rostova	Enterprise	DE	1	5900	2026-02-05 16:45:00
CUS-1002	Lucas Silva	Enterprise	BR	1	3640	2026-01-22 11:00:00
CUS-1005	Sophie Martin	Growth	FR	1	1620	2026-02-28 15:30:00
CUS-1004	Tariq Mansour	Growth	AE	1	1440	2026-01-18 10:20:00
CUS-1008	Amina Diallo	Startup	SN	1	680	2026-03-01 17:10:00
CUS-1007	Liam O Connor	Startup	IE	1	480	2026-02-10 12:00:00
CUS-1006	Kenji Sato	Growth	JP	0	0	NULL

Multiple connections. Separate context. Clear visual identity.

Each connection keeps its own tabs, schema, results and QuerIA context.

**DEMO PROD****DEMO DEV****DEMO LOCAL**

Names, colors and environment levels are configured by the user.

Connection identity is explicit.

The user names the connection, chooses its color and sets the environment level.

Edit connection
Local SQLite database file

General Advanced Diagnostics

Name
DEMO PROD

Driver
SQLite

Database file
Browse...

Folder
Choose or create a folder

Read-only mode

Environment
✓ Production with extra safeguards
Development
Local

Color
[Red bar]

User-configured environment

Production, Development and Local are explicit choices, not automatic detection.

Visual identity

Colors make the active connection obvious while working across tabs.

Read-only option

A connection can be marked read-only when writes should never happen.

Safety rules match the operation.

Confirmation policies can be different for writes, DDL, restores and server actions.

Preferences

LakeDB behavior

General

Queries

Confirmations

History

Account

Operation	Production	Development	Local
INSERT, UPDATE and other writes Confirms INSERT, UPDATE, REPLACE, LOAD and other data-changing SQL.	Level 1 · Button	Level 0 · No confirmation	Level 0 · No confirmation
DELETE statements and row deletions Confirms every DELETE, even with WHERE or when deleting rows from a grid.	Level 1 · Button	Level 0 · No confirmation	Level 0 · No confirmation
ALTER statements Confirms ALTER TABLE and other ALTER operations.	Level 2 · Type word	Level 0 · No confirmation	Level 0 · No confirmation
DROP statements Confirms DROP TABLE, DROP DATABASE and other DROP operations.	Level 2 · Type word	Level 0 · No confirmation	Level 0 · No confirmation
TRUNCATE statements Confirms every TRUNCATE operation.	Level 2 · Type word	Level 0 · No confirmation	Level 0 · No confirmation
Other structure and permission changes Confirms CREATE, RENAME, GRANT, REVOKE and other DDL.	Level 2 · Type word	Level 0 · No confirmation	Level 0 · No confirmation
Replace views, routines, triggers and events Confirms DROP and CREATE when applying an edited object definition.	Level 2 · Type word	Level 0 · No confirmation	Level 0 · No confirmation
Table and result inserts or updates Confirms inserts and updates prepared in table or result grids.	Level 1 · Button	Level 0 · No confirmation	Level 0 · No confirmation
Migration, copy and SQL restore tools Confirms operations that write through Database tools.	Level 2 · Type word	Level 0 · No confirmation	Level 0 · No confirmation
Stop server queries Confirms interrupting a running database session.	Level 1 · Button	Level 0 · No confirmation	Level 0 · No confirmation

Cancel

Save

Policy levels

- Level 0: no confirmation
- Level 1: button confirmation
- Level 2: type-word confirmation
- Separate rules per environment
- Editable without changing database metadata

Visible before use

The matrix explains exactly what LakeDB will ask for before a risky action runs.

High-risk actions can demand stronger confirmation.

A destructive operation does not continue until the configured token is entered.

Delete table

 The table main.audit_events and all its data will be permanently deleted.

Type PRODUCTION to confirm

Cancel

Delete table

Production safeguard

- The table is named before deletion.
- The required token is shown explicitly.
- Cancel remains the safe default.

Type PRODUCTION

The policy comes from the connection and operation configuration.

A complete SQL workflow before AI.

Write, run and inspect real queries in one workspace.

The screenshot displays the LakeDB SQL editor interface. At the top, a tab labeled 'Query 1' is active. Below it, the 'Active database' is set to 'main'. The SQL editor contains the following query:

```
1 SELECT
2   c.customer_ref,
3   c.full_name,
4   c.segment,
5   c.country,
6   COUNT(o.id) AS order_count,
7   ROUND(COALESCE(SUM(o.total_amount), 0), 2) AS lifetime_value,
8   MAX(o.ordered_at) AS last_order_at
9 FROM customers c
10 LEFT JOIN orders o ON o.customer_id = c.id
11 GROUP BY
12   c.id,
13   c.customer_ref,
14   c.full_name,
15   c.segment,
16   c.country
17 ORDER BY
18   lifetime_value DESC,
19   c.full_name
20 LIMIT 8;
```

Below the editor, the 'Query results' tab is active, showing a table with 7 columns: customer_ref, full_name, segment, country, order_count, lifetime_value, and last_order_at. The table contains 8 rows of data.

customer_ref	full_name	segment	country	order_count	lifetime_value	last_order_at
CUS-1001	Maya Chen	Enterprise	US	2	9820	2026-02-14 14:15:00
CUS-1003	Elena Rostova	Enterprise	DE	1	5900	2026-02-05 16:45:00
CUS-1002	Lucas Silva	Enterprise	BR	1	3640	2026-01-22 11:00:00
CUS-1005	Sophie Martin	Growth	FR	1	1620	2026-02-28 15:30:00
CUS-1004	Tariq Mansour	Growth	AE	1	1440	2026-01-18 10:20:00
CUS-1008	Amine Diallo	Startup	SN	1	680	2026-03-01 17:10:00
CUS-1007	Liam O Connor	Startup	IE	1	480	2026-02-10 12:00:00
CUS-1006	Kenji Sato	Growth	JP	0	0	NULL

The basic workflow matters first

- SQL editor and results share the same workspace.
- Results show real rows from the connected database.
- Duration and row count stay visible after execution.
- No AI is required for everyday SQL work.

SQL editor

Query results

Duration + rows

Schema-aware autocomplete stays close to the editor.

Tables, views, indexes and columns appear where they are needed.

Tables and views

Active database **main**

1 **SELECT** * **FROM**

audit_events

customers

legacy_import_queue

order_items

orders

products

active_product_catalog

customer_order_summary

Table · main · Alias a

Indexes and columns in WHERE

1 **SELECT** * **FROM** orders o **WHERE** o.

Use index PRIMARY Unique index

id

Use index sqlite_autoindex_orders_1 Unique ind...

order_r...

id PK

INTEGER

order_ref UQ sqlite_autoindex_orders_1

TEXT

customer_id

INTEGER

status

TEXT

total_amount

REAL

ordered_at

TEXT

Context from metadata

LakeDB uses the schema you are connected to, not a generic SQL template.

Less typing, fewer mistakes

Alias, column and index suggestions reduce friction while writing queries.

No AI required

The editor remains useful even with QuerIA disabled.

Explain before optimizing.

Plan insights stay next to the SQL that produced them.

The screenshot shows the LakeDB interface with a SQL query editor and a results pane. The query is as follows:

```
1 SELECT
2   c.customer_ref,
3   c.full_name,
4   c.segment,
5   c.country,
6   COUNT(o.id) AS order_count,
7   ROUND(COALESCE(SUM(o.total_amount), 0), 2) AS lifetime_value,
8   MAX(o.ordered_at) AS last_order_at
9 FROM customers c
10 LEFT JOIN orders o ON o.customer_id = c.id
11 GROUP BY
12   c.id,
13   c.customer_ref,
14   c.full_name,
15   c.segment,
16   c.country
17 ORDER BY
18   lifetime_value DESC,
19   c.full_name
20 LIMIT 8;
```

The results pane shows the execution plan for the query:

id	parent	notused	detail
10	0	211	SCAN c USING INDEX sqlite_autoindex_cust...
15	0	0	BLOOM FILTER ON o (customer_id=?)
27	0	53	SEARCH o USING AUTOMATIC COVERING INDEX ...
93	0	0	USE TEMP B-TREE FOR ORDER BY

Plan review

- Run EXPLAIN without leaving the editor.
- See scans, indexes and sort operations.
- Keep the plan tied to the current query.

SQL first

Plan visible

Teach the formatter your style.

Learn a deterministic profile from explicit examples you provide.

Learn formatting style from examples

Local deterministic analysis. Examples stay on this device.

SELECT

INSERT

UPDATE

DELETE

SELECT

```
SELECT
  u.id,
  u.email,
  u.first_name,
  u.last_name,
  u.status,
  p.display_name,
  p.country,
  COUNT(o.id) AS total_orders,
  SUM(o.total_amount) AS total_spent,
  MAX(o.created_at) AS last_order_at
FROM users u
  JOIN profiles p
    ON p.user_id = u.id
  LEFT JOIN orders o
    ON o.user_id = u.id
WHERE u.active = 1
  AND u.deleted_at IS NULL
  AND p.country IS NOT NULL
  AND (
    o.status = 'completed'
    OR o.status = 'processing'
  )
GROUP BY
  u.id,
  u.email,
  u.first_name,
  u.last_name,
  u.status,
  p.display_name,
  p.country
HAVING COUNT(o.id) > 0
ORDER BY total_spent DESC, u.email ASC
LIMIT 100;
```

Detected rules

- ✓ Indentation: 2 spaces
- ✓ Keywords: uppercase
- ✓ Commas: after continued items
- ✓ Major clauses: new lines
- ✓ Columns: one per line
- ✓ JOIN / ON: ON on a new line
- ✓ JOIN indentation: one level below FROM
- ✓ ON indentation: one level below JOIN
- ✓ WHERE AND / OR: new lines
- ✓ GROUP BY columns: one per line
- ✓ ORDER BY columns: single line

No examples are sent to QuerIA or any external service.

SELECT

INSERT

UPDATE

DELETE

Local · deterministic · no external request

Formatting does not consume AI tokens or send SQL to QuerIA.

Apply the saved style to real SQL.

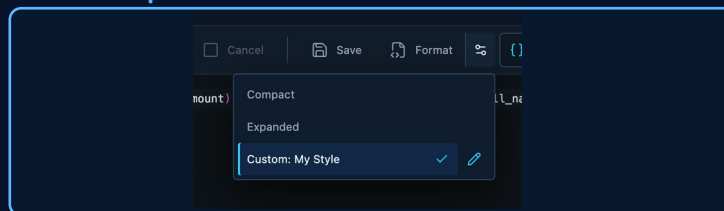
Use Custom: My Style whenever you format the editor.

Before: one-line SQL

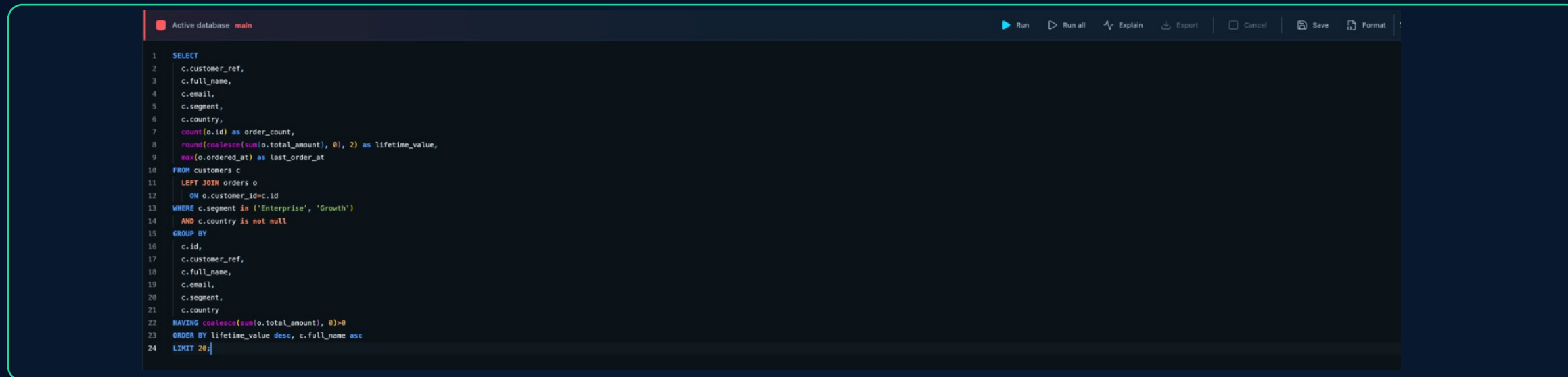


The screenshot shows a code editor with a dark theme. The top toolbar includes buttons for Run, Run all, Explain, Export, Cancel, Save, Format, and Snippets. The main text area contains a single-line SQL query: `1 ed_at) as last_order_at from customers c left join orders o on o.customer_id=c.id where c.segment in ('Enterprise','Growth') and c.country is not null group by c.id,c.customer_ref,c.full_name,c.email,c.segment,c.country having coalesce(sum(o.total_amount),0)>0 order by lifetime_value desc,c.full_name asc limit 20;`

Formatter profile



After: Custom My Style



The screenshot shows the same code editor as before, but the SQL query is now formatted into a multi-line, readable structure. The query is:

```
1 SELECT
2   c.customer_ref,
3   c.full_name,
4   c.email,
5   c.segment,
6   c.country,
7   count(o.id) as order_count,
8   round(coalesce(sum(o.total_amount), 0), 2) as lifetime_value,
9   max(o.ordered_at) as last_order_at
10 FROM customers c
11 LEFT JOIN orders o
12   ON o.customer_id=c.id
13 WHERE c.segment in ('Enterprise', 'Growth')
14 AND c.country is not null
15 GROUP BY
16   c.id,
17   c.customer_ref,
18   c.full_name,
19   c.email,
20   c.segment,
21   c.country
22 HAVING coalesce(sum(o.total_amount), 0)>0
23 ORDER BY lifetime_value desc, c.full_name asc
24 LIMIT 20;
```

QuerIA proposes. You remain in control.

Generate SQL from natural language, review it, and run it only when you decide.

Query 1
QuerIA 1

Active database: main

Read only

Generate with AI

Normal

Agentic

CALIMACO

Save

Copy SQL

1 Show each customer with their name, segment, country, number of completed orders, total amount spent and most recent order date. Include customers with no orders, sort by total spent descending and limit the result to 20 customers
2 **Agent** QUERY: SELECT "c"."full_name", "c"."segment", "c"."country", COUNT("o"."id") AS "number_of_completed_orders", COALESCE(SUM("o"."total_amount"), 0) AS "total_amount_spent", MAX("o"."ordered_at") AS "most_recent_order_date" FROM "main"."customers" AS "c" LEFT JOIN "main"."orders" AS "o" ON "c"."id" = "o"."cust...

The query uses a LEFT JOIN to ensure all customers are included. It filters 'COMPLETED' orders to calculate the required metrics. COALESCE ensures that customers with no orders show 0 spent instead of NULL.

```

SELECT
  "c"."full_name",
  "c"."segment",
  "c"."country",
  COUNT("o"."id") AS "number_of_completed_orders",
  COALESCE(SUM("o"."total_amount"), 0) AS "total_amount_spent",
  MAX("o"."ordered_at") AS "most_recent_order_date"
FROM "main"."customers" AS "c"
LEFT JOIN "main"."orders" AS "o"
  ON "c"."id" = "o"."customer_id"
 AND "o"."status" = 'COMPLETED'
GROUP BY "c"."id"
ORDER BY "total_amount_spent" DESC
LIMIT 20;

```

Assumptions: The status value for completed orders is 'COMPLETED'. - Total amount spent is the sum of total_amount from orders marked as 'COMPLETED'.

Tables and indexes

Plan validated with EXPLAIN

o

Access: SCAN No index used

o

Access: SEARCH No index used

Run query

Copy SQL

main.customers

main.orders

Results stay on this device

Query results

order_items

orders

customers

Results

Messages

History

DDL

Errors

full_name

segment

country

number_of_completed_orders

total_amount_spent

most_recent_order_date

Maya Chen

Enterprise

US

0

0

NULL

Ask

Generate

Review SQL

Run when ready

Nothing runs automatically.

AI suggestions remain visible and reviewable before execution.

Keep the database error. Add help on top.

Preserve the engine error, then optionally review a corrected SQL proposal.

Original error and failed statement

The screenshot shows the LakeDB interface with a dark theme. At the top, there are tabs for 'Query results', 'order_items', 'orders', and a search bar. Below the tabs, there are tabs for 'Results', 'Messages', 'History', 'DDL', and 'Errors'. The 'Errors' tab is selected, showing a message: 'Statement 2 failed SQLITE_ERROR · Database error DEMO PROD · main 3 ms · 18/8/2026, 12:25:58'. Below this, the error message 'no such column: customer_name' is displayed. A button 'Go to statement' is on the right, and a button 'Explain and fix with AI' is at the bottom left.

Optional AI SQL correction

The screenshot shows the LakeDB interface with a dark theme. At the top, there are tabs for 'Query results', 'order_items', 'orders', 'customers', and a search bar. Below the tabs, there are tabs for 'Results', 'Messages', 'History', 'DDL', and 'Errors'. The 'Errors' tab is selected, showing a message: 'Statement 2 failed SQLITE_ERROR · Database error DEMO PROD · main 3 ms · 18/8/2026, 12:43:43'. Below this, the error message 'no such column: customer_name' is displayed. A button 'Go to statement' is on the right. Below the error message, there is a section titled 'AI SQL correction' with a close button. It contains two SQL queries: 'SELECT COUNT(*) AS "customer_count" FROM "main"."customers";' and 'SELECT "customer_ref", "full_name" FROM "main"."customers" LIMIT 5;'. A button 'Copy SQL' is at the bottom left. To the right of the SQL queries, there is a section titled 'What happened' with the text: 'The column 'customer_name' does not exist in the 'customers' table. The query also lacked proper double-quoting of identifiers required by the dialect instructions.' Below this, there is a section titled 'Correction' with the text: 'Removed the non-existent 'customer_name' column and applied double quotes to identifiers and qualified the table name.'

Original error visible

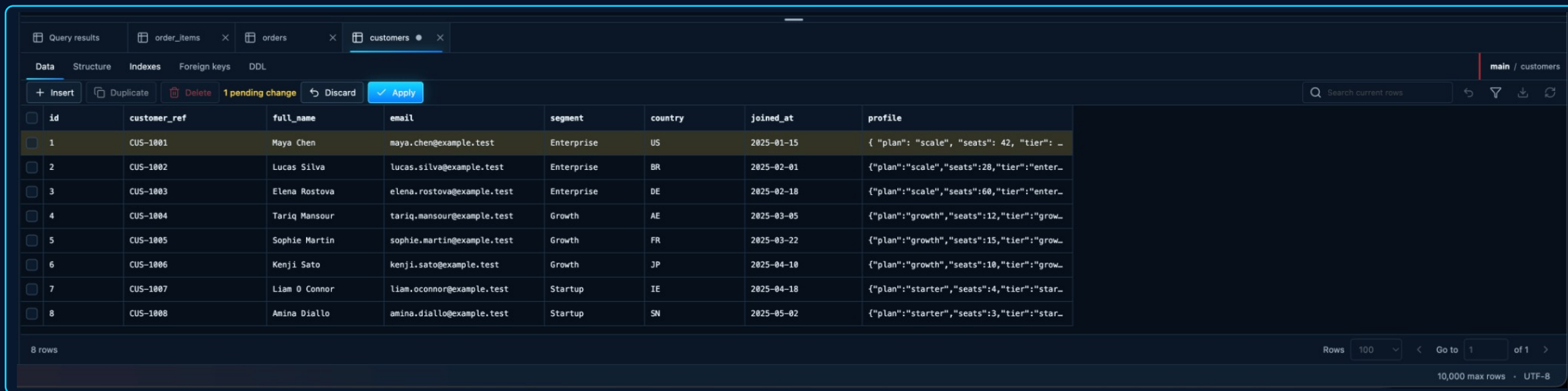
Go to statement

Copy reviewed SQL

No auto-execution

Edit rows without losing control.

Buffered changes remain visible until you explicitly apply or discard them.



id	customer_ref	full_name	email	segment	country	joined_at	profile
1	CUS-1001	Maya Chen	maya.chen@example.test	Enterprise	US	2025-01-15	{ "plan": "scale", "seats": 42, "tier": ...
2	CUS-1002	Lucas Silva	lucas.silva@example.test	Enterprise	BR	2025-02-01	{ "plan": "scale", "seats": 28, "tier": "enter...
3	CUS-1003	Elena Rostova	elena.rostova@example.test	Enterprise	DE	2025-02-18	{ "plan": "scale", "seats": 60, "tier": "enter...
4	CUS-1004	Tariq Mansour	tariq.mansour@example.test	Growth	AE	2025-03-05	{ "plan": "growth", "seats": 12, "tier": "grow...
5	CUS-1005	Sophie Martin	sophie.martin@example.test	Growth	FR	2025-03-22	{ "plan": "growth", "seats": 15, "tier": "grow...
6	CUS-1006	Kenji Sato	kenji.sato@example.test	Growth	JP	2025-04-10	{ "plan": "growth", "seats": 10, "tier": "grow...
7	CUS-1007	Liam O Connor	liam.oconnor@example.test	Startup	IE	2025-04-18	{ "plan": "starter", "seats": 4, "tier": "star...
8	CUS-1008	Amina Diallo	amina.diallo@example.test	Startup	SN	2025-05-02	{ "plan": "starter", "seats": 3, "tier": "star...

One pending change

The grid marks unsaved edits before they reach the database.

Apply explicitly

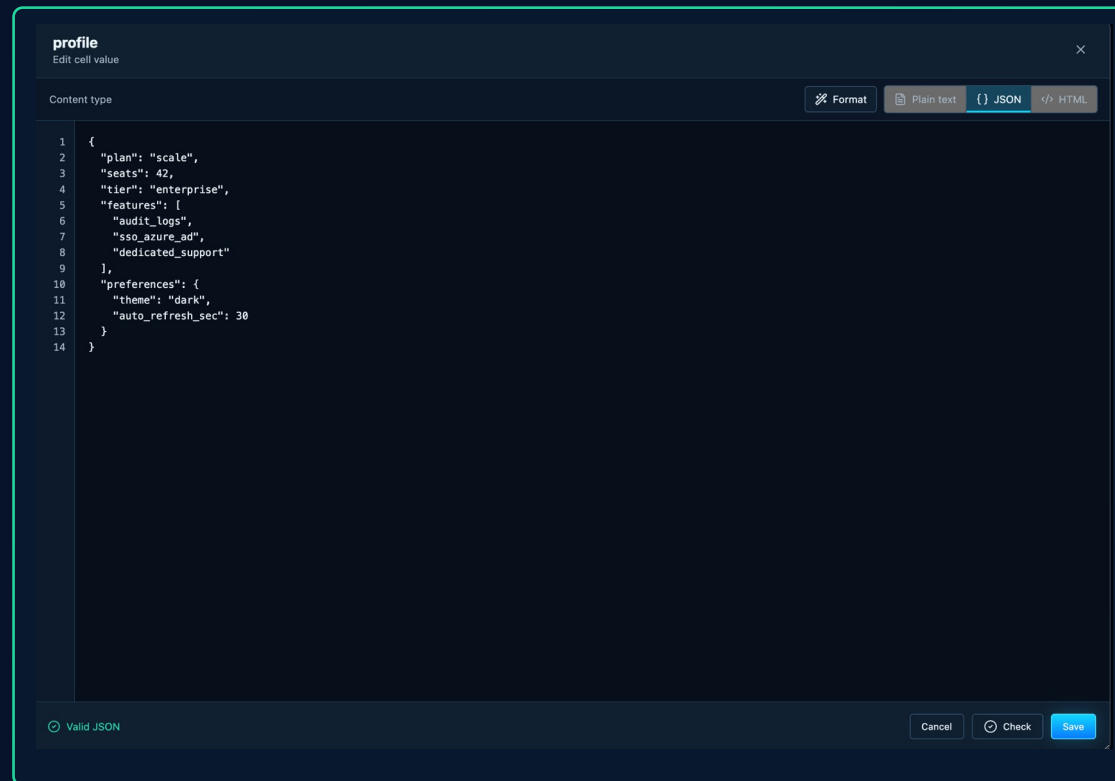
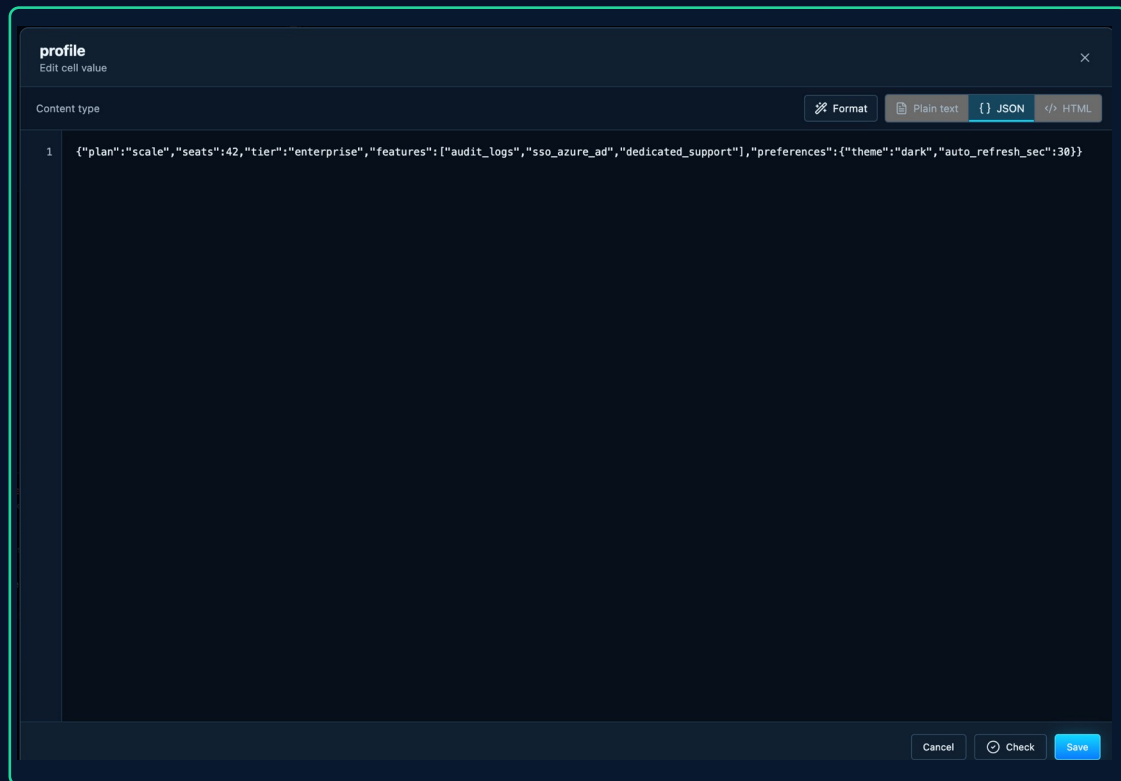
Commit buffered changes only when the edited row is ready.

Discard safely

Return to the last persisted state without manual rollback SQL.

Format structured values before saving.

A dedicated editor makes JSON readable without leaving the table workflow.



Format

Check

Valid JSON

Structured values remain visible and reviewable before save.

Validation feedback stays precise.

Invalid JSON is blocked with the exact position of the problem.

Invalid JSON

Exact line + column

Fix before save

profile

Edit cell value

Content type

Format

Plain text

JSON

HTML

```
1 {
2   "plan": "scale",
3   "seats": 42,
4   "tier": "enterprise",
5   "features": [
6     "audit_logs",
7     "sso_azure_ad"
8     "dedicated_support"
9   ],
10  "preferences": {
11    "theme": "dark",
12    "auto_refresh_sec": 30
13  }
14 }
```

Invalid JSON: Expected ',' or ']' after array element in JSON at position 117 (line 8 column 5)

Cancel

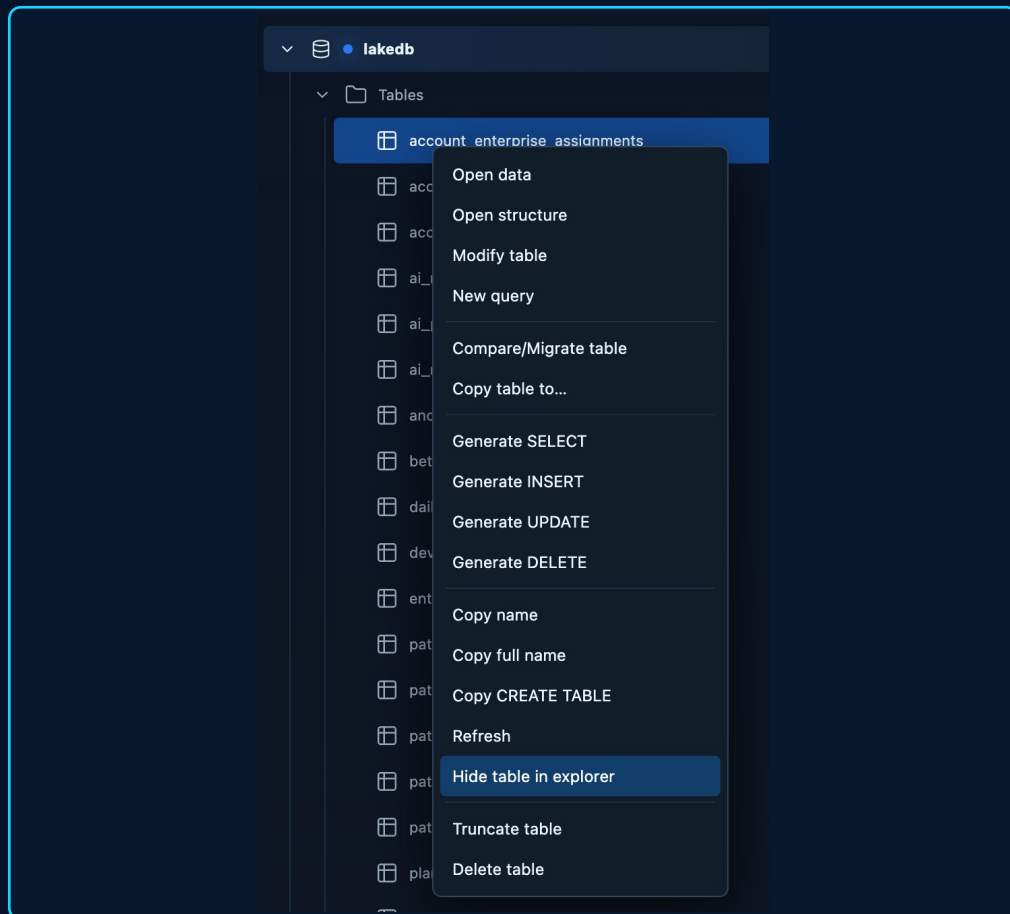
Check

Save

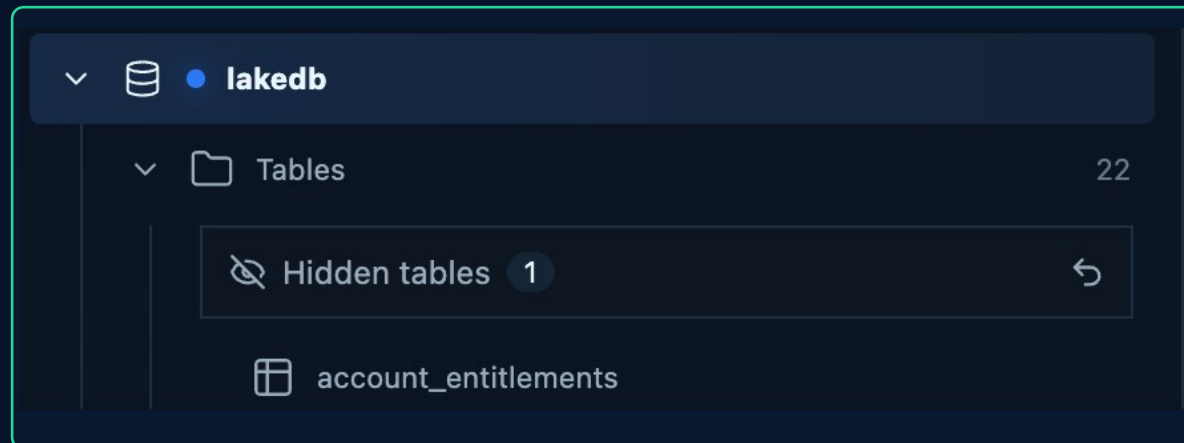
Keep the object explorer focused.

Hide noisy tables locally and restore them without changing database metadata.

Hide from the table context menu



Restore from Hidden tables



Local explorer preferences

- Hide auxiliary or noisy tables from the tree.
- Restore them later without changing the database.

Hide locally

Restore quickly

Inspect active database sessions when needed.

Server activity exposes running processes and operational controls on supported engines.

Server activity

Inspect database sessions and replication status

Processes 3

Replication

Connection

OVH DLago · MySQL · 127.0.0.1:3306

Search sessions

Refresh

ID	User	Database	Time	State	SQL	
41580	david_lakedb@172.18.0.1:4...	lakedb	8,027 s		-	KILL
42407	david_lakedb@172.18.0.1:4...	lakedb	13 s		-	KILL
42408	david_lakedb@172.18.0.1:4...	lakedb	0 ms	init	SHOW FULL PROCESSLIST	KILL

Disconnect
Reconnect
Refresh
Server activity
Backup, restore and migrations...
Execute
Cancel query

Demo MySQL

×

database demo

Processes

Inspect active sessions, duration and the SQL currently running.

Kill / cancel

Stop a selected server session when the workflow requires it.

Replication

Check replication status from the same operational view.



Local client first. Enterprise is optional.

Core database work stays local and free. Enterprise can add isolated services and closer engineering collaboration.

LakeDB desktop client

- Connections, workspaces and SQL editor
- Autocomplete, results and data editing
- Backup, restore, formatting and safety tools
- Free to use; QuerIA includes a free allowance

Optional Enterprise setup

- Dedicated QuerIA service
- Organization controls and isolated configuration
- Company-owned provider settings and limits
- Closer engineering collaboration when useful

Try it with a real workflow and tell me what gets in your way.